

1. Introduction to Regression

Regression is a supervised learning technique used to predict continuous numerical values by learning relationships between input variables (features) and an output variable (target). It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.

Regression differs from classification in the nature of its output. **While regression** predicts a continuous numeric quantity, **classification** predicts a discrete category or class label. In other words, **regression answers “how much” or “how many,” while classification answers “which category.”** Both are forms of supervised learning, since both rely on labeled training data, but the type of target variable determines which approach is appropriate for a given problem.

For example, predicting the price of a car based on its mileage, age, and brand is a regression problem, since the output (price) is a continuous number that can take any value within a range. In contrast, predicting whether an email is spam or not spam is a classification problem, since the output is one of two fixed categories rather than a number on a continuous scale.

2. Types of Regression

Regression can be classified into different types based on the number of predictor variables and the nature of the relationship between variables:

2.1 Simple Linear Regression

Simple Linear Regression models the relationship between one independent variable and a continuous dependent variable by fitting a straight line that minimizes the sum of squared errors. It assumes a constant rate of change, meaning the output varies proportionally with the input.

- Application: Estimating car price from only its model
- Advantage: Highly interpretable due to its simple mathematical structure
- Disadvantage: Cannot capture curved or complex data patterns

2.2 Multiple Linear Regression

Multiple Linear Regression extends simple linear regression by incorporating multiple independent variables to predict a continuous outcome. Each predictor is assigned a coefficient that reflects its individual impact while holding other variables constant.

- Application: Predicting car prices using multiple factors like model, year and number of seats
- Advantage: Captures the combined influence of many factors simultaneously
- Disadvantage: Performance drops in the presence of multicollinearity (features highly correlated with each other)

2.3 Polynomial Regression

Polynomial Regression models non-linear relationships by transforming input features into higher-degree polynomial terms (e.g. x^2 , x^3). Although it models non-linear relationships in input features, it is linear in coefficients (parameters), which is why it is still considered a linear model.

- Application: Modelling curved growth trends like population increase or temperature variation
- Advantage: Effectively captures non-linear relationships without switching to non-linear algorithms
- Disadvantage: Higher-degree polynomials may lead to overfitting and unstable predictions

3. Regression Metrics

Evaluation in machine learning measures the performance of a model. The following metrics are commonly used to evaluate regression models:

- Mean Absolute Error (MAE): The average absolute difference between the predicted and actual values of the target variable.
- Mean Squared Error (MSE): The average squared difference between the predicted and actual values of the target variable.
- Root Mean Squared Error (RMSE): The square root of the mean squared error, expressed in the same unit as the target variable.
- R^2 Score (Coefficient of Determination): The proportion of variance in the target variable that is explained by the model, relative to simply predicting the mean. Values typically range from 0 to 1, with higher values indicating a better fit.

These four metrics are related but emphasize different aspects of model performance. MAE treats every error equally regardless of magnitude, while MSE and RMSE square the errors first, which penalizes larger mistakes far more heavily than smaller ones. R^2 , unlike the other three, is not an error measure in the same units as the target; instead, it expresses how much better the model performs compared to a naive baseline that always predicts the mean. The table below summarizes the key differences between the four metrics.

Metric	What it measures	Unit	Sensitivity to large errors	Typical use
MAE	Average absolute distance between predicted and actual values	Same as target	Low — all errors weighted equally	Reporting performance in an intuitive, equally-weighted way
MSE	Average of squared differences between predicted and actual values	Target squared	High — large errors penalized quadratically	Training/optimization, since it is smooth and differentiable
RMSE	Square root of MSE, bringing the error back to the original unit	Same as target	High — still penalizes large errors heavily	Reporting performance when large mistakes should be discouraged
R^2	Proportion of variance in the target explained by the model, relative to predicting the mean	Unitless (0 to 1 typically)	Moderate — large errors reduce it, but it is bounded and relative	Comparing overall model fit and explanatory power

4. Underfitting and Overfitting

Underfitting occurs when a model is too simple to capture the underlying pattern in the data. An underfit model performs poorly on both the training data and the test data, because it has not learned enough structure from the data to make accurate predictions in the first place.

Overfitting occurs when a model is too complex relative to the amount or simplicity of the underlying pattern, causing it to memorize the training data, including its noise and random fluctuations, rather than learning the true relationship. An overfit model performs very well on training data but poorly on unseen test data, since the patterns it learned do not generalize.

In polynomial regression specifically, overfitting becomes more likely as the degree of the polynomial increases. A higher-degree polynomial has enough flexibility to bend through almost every individual training point, including outliers and noise, instead of capturing the smoother underlying trend. The result is a curve that fits the training data extremely well but oscillates wildly and predicts poorly on new data points.

Several methods can help prevent overfitting:

- **Limiting model complexity:** for example, reducing the polynomial degree in polynomial regression, or constraining a decision tree with parameters such as `max_depth` and `min_samples_leaf`, so the model is forced to learn general patterns rather than memorize individual data points.
- **Using cross-validation:** splitting the training data into multiple folds and evaluating performance across them helps detect overfitting before final testing, since a model that overfits will show inconsistent performance across folds.
- **Applying regularization:** techniques such as Ridge or Lasso regression add a penalty term that discourages overly large coefficients, which keeps the model simpler and less sensitive to noise.
- **Using ensemble methods:** approaches such as Random Forest combine many individual decision trees and average their predictions, which reduces the overfitting tendency of any single tree.

5. Real-World Case Study

Predicting Used Car Prices Using Multiple Linear Regression and Random Forest

A widely referenced application of regression in the automotive domain is the prediction of used car prices based on vehicle attributes such as mileage, age, brand, fuel type, and engine size. Several published machine learning studies on used-car marketplaces (for example, analyses based on datasets such as the UCI “Automobile” dataset and various Kaggle used-car price datasets) follow a very similar approach to the one implemented in Part B of this assignment.

Goal: The objective of these studies is typically to build a model that can estimate a fair market price for a used car given its specifications, helping buyers avoid overpaying and helping sellers price their vehicles competitively.

Data used: The datasets generally include structured tabular features such as brand, model year, mileage (odometer reading), fuel type, transmission type, engine size, and number of previous owners, with the sale price as the target variable. This mirrors the structure of the cleaned car dataset used in this assignment, where Price was predicted from a set of cleaned categorical and numerical vehicle features.

Type of regression applied: Most of these studies begin with Multiple Linear Regression as a simple, interpretable baseline model, since price is assumed to depend on several independent factors simultaneously. They then commonly extend the analysis with more flexible models such as Random Forest or Gradient Boosting, since the relationship between vehicle features and price is rarely perfectly linear (for instance, depreciation with mileage is often non-linear, dropping sharply in the first few years and then levelling off).

Key results and insights: Across these studies, ensemble methods such as Random Forest consistently outperform plain Linear Regression on standard metrics such as RMSE and R^2 , because they can capture non-linear interactions between features (for example, the combined effect of low mileage and a premium brand) that a linear model cannot represent with a single coefficient per feature. This finding closely matches the results obtained in Part B of this assignment, where the Random Forest model produced notably better R^2 and lower error metrics than the Linear Regression model on the cleaned car dataset, reinforcing the broader pattern observed in real-world used-car price prediction research.

References

- GeeksforGeeks. Linear Regression in Machine Learning. <https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>
- GeeksforGeeks. Multiple Linear Regression using Python. <https://www.geeksforgeeks.org/machine-learning/ml-multiple-linear-regression-using-python/>
- GeeksforGeeks. Polynomial Regression from Scratch using Python. <https://www.geeksforgeeks.org/machine-learning/polynomial-regression-from-scratch-using-python/>
- GeeksforGeeks. How to Calculate Mean Absolute Error in Python. <https://www.geeksforgeeks.org/python/how-to-calculate-mean-absolute-error-in-python/>
- GeeksforGeeks. Python | Mean Squared Error. <https://www.geeksforgeeks.org/python/python-mean-squared-error/>
- GeeksforGeeks. RMSE – Root Mean Square Error. <https://www.geeksforgeeks.org/software-engineering/rmse-root-mean-square-error-in-matlab/>
- GeeksforGeeks. Python | Coefficient of Determination – R^2 Score. <https://www.geeksforgeeks.org/machine-learning/python-coefficient-of-determination-r2-score/>